PRM393 – MOBILE PROGRAMMING · LAB 3

EXEMatching: Cloud-Enabled Mobile App

Firebase Cloud Backend · Google ARTEMIS Automated Testing · AI-Assisted Code Review

DEVELOPMENT TEAM: DEVORGANIZE222
Platform: Flutter (Android API 36 / Pixel API 36)

Firebase ID: prm393-lab-3-exematching
Package: com.devorganize222.exematching

Topic, Persona & Three Critical User Flows

TARGET PERSONA & SUCCESS CRITERION

- Persona: Minh Le – senior Software Engineering student facing capstone registration deadline.
- Problem: Difficulty finding matching teammates, coordinating leader election, and recovering if a member drops.
- Success Criterion: Fast team discovery, confidential leader election, and seamless recovery via Waiting Pool to lock roster before deadline.

THREE CRITICAL FLOWS IMPLEMENTED

- Flow 1 (Discover & Join): Search with skill chips -> View details -> Join open role -> Concurrency check.
- Flow 2 (Leader Voting): Confidential voting -> Real-time tally reveal upon majority -> Confirmation dialog.
- Flow 3 (Roster Finalization): Leader validates criteria (4-5 members) -> Recovers via Waiting Pool -> Locks team.

Screen Implementation & UX Deviations

10 PRODUCTION SCREENS BUILT

- [x] SCR_01 Sign In: Google SSO with allowed roster check.
- [x] SCR_02 Dashboard: Active team status & countdown banner.
- [x] SCR_03 Browse: Debounced search, skill chips, empty state.
- [x] SCR_04 Team Detail: Role selection & atomic join modal.
- [x] SCR_05 My Team: Roster management & dynamic action button.
- [x] SCR_06 Leader Voting: Hidden tallies & majority election.
- [x] SCR_07 Lock Review: Capacity check & Waiting Pool recovery.
- [x] SCR_08 Random Pool: Queue status & estimated match time.
- [x] SCR_09 Alerts: Inbox with Decline + Undo support.
- [x] Profile Screen: Firebase Demo Area & Cloud Export.

KEY HEURISTIC DEVIATIONS (docs/design-traceability.md)

- [x] DV-01: Unified "Group" and "Team" to "Team" everywhere (UI Consistency Heuristic).
- [x] DV-04: Replaced unhandled join "request" with direct open-slot join validated by backend transaction.
- [x] DV-05: Front-loaded error messages ("Can't lock yet: needs 1 more member" instead of vague technical codes).
- [x] DV-08: Added Profile screen as single destination for user profile and Firebase verification actions.
- [x] DV-09/10: Elevated contrast to exceed WCAG AA standards (buttons >=48dp, text >=14sp, colors >=4.5:1).

MVVM Architecture & Layer Separation

STRICT DOWNWARD DEPENDENCIES

- View Layer (lib/screens/ & lib/widgets/): Pure declarative widgets, zero SDK calls, zero business logic.
- ViewModel Layer (lib/viewmodels/): ChangeNotifier owning ViewState<T>, validation rules, and analytics triggers.
- Service Layer (lib/services/): Abstract interfaces defining cloud and data contracts.
- Data/Cloud Layer (lib/firebase/): Concrete Firebase SDK implementations encapsulated entirely.
- Composition Root (lib/app/providers.dart): Single file injecting singletons into ViewModels via Provider.

EXPLICIT STATE MODELING (ViewState<T>)

- Loading: Renders skeleton list or non-blocking spinner.
- Empty: Renders informative illustration, clear message, and single recovery action (e.g. "Clear Filters").
- Failure: Contains AppFailure with title, cause, and retryable flag -> shows retry button (never crashes).
- Data: Renders populated domain models.
- Unit-Testable: ViewModels can be tested with fake service implementations without running emulator.

Design System in Code & Reusable Widget Library

CENTRALIZED DESIGN TOKENS (lib/theme/)

- ColorScheme: Material 3 scheme with custom FPT brand tokens (Primary #0284C7, Success #15803D, Warning #B4530A).
- TextTheme: Strict typography hierarchy using Roboto (system font on Android API 36).
- AppSpacing: Named constants xs (4), sm (8), md (16), lg (24), xl (32).
- AppRadius: Named corner radiuses sm (4), md (8), lg (12), pill (999).
- AppElevation: Low elevation tokens for modern flat design.

REUSABLE COMPONENT LIBRARY (lib/widgets/)

- AppButton: Primary, secondary, destructive, text with loading overlay and 48dp minimum touch target.
- AppTextField: Styled search and form inputs with focused borders and clear actions.
- AppCard: Standard elevated card and SelectableCard with radio check indicator.
- Status Indicators: StatusBadge, InfoBanner, SlotRow, AppProgressBar (icon + label, never color alone).
- AppNavigationShell: Custom bottom navigation with real-time unread notification badge.

Firebase Authentication & Roster Authorization

AUTHENTICATION IMPLEMENTATION

- Google Sign-In: Native Google credential sign-in via FirebaseAuth.instance.signInWithCredential().
- Session Stream: Directly subscribes to authStateChanges() for immediate login/logout state routing.
- Sign-Out Cleanup: Explicitly signs out of both Google and Firebase, purging user memory cache.
- Cancellation & Offline: User cancellation or network drop emits clean AppFailure with user guidance.

CENTRALIZED ROSTER ALLOWLIST (K19+ SOLUTION)

- Problem: FPT students from K19+ use non-standard email domains, requiring flexible yet secure roster validation.
- Solution: Firestore collection allowedEmails/{email} stores authorized class roster with enabled: true.
- Two-Tier Check: Client-side _studentFor() verifies email against roster; Security Rules enforce isRosterMember().
- Unauthorized Denial: Non-roster Google accounts are immediately rejected with "This is not an FPT account" (TC-01).

Cloud Firestore Transactions & Security Rules

CONCURRENT TRANSACTIONS (firestore_team_repository.dart)

- Atomic Roster Mutations: Join, leave, vote, lock, and pool-invite run in runTransaction.
- Double-Join Prevention: Validates memberIds before committing; concurrent joins safely abort and retry.
- Leader Election: Tallies remain hidden in UI until member votes; election declares winner at majority (>50%).
- Waiting Pool Recovery: Atomic assignment of candidate from waitingPool to team in a single transaction.

FIRESTORE SECURITY RULES (firebase/firestore.rules)

- Roster Gating: All queries require isRosterMember() (email in allowedEmails).
- Whitelist Update Fields (GR-01 Fix): Students can ONLY modify 8 roster fields (members, votes, leaderId, lockCode).
- Immutability: Team metadata (supervisor, capacity, topic) cannot be tampered with by clients.
- User Isolation: students/{uid} and notifications readable/writable exclusively by resource owner.

CLOUD SERVICE 03

Firebase Cloud Storage & User Isolation

REPORT GENERATION & UPLOAD (firebase_storage_service.dart)

- PDF Synthesis: PdfReportService renders formal team summary PDF matching Figma design tokens.
- Per-User Path: Automatically uploaded to reports/{uid}/{fileName} in Firebase Cloud Storage.
- Real-Time Progress: Emits UploadProgress stream updating visual progress bar from 0% to 100%.
- Download Verification: Retrieves and displays signed download URL upon upload completion.

STRICT STORAGE SECURITY RULES (firebase/storage.rules)

- Strict Isolation: allow read, write: if request.auth != null && request.auth.uid == userId;
- No Open Rules: Never uses allow read, write: if true; (which would cause zero marks on storage).
- Cross-User Denial: Student A is strictly forbidden from reading or overwriting Student B's reports.
- Console Evidence: Bucket prm393-lab-3-exematching.firebasestorage.app holds verified PDF uploads.

Firebase Cloud Messaging Across 3 Lifecycle States

LIFECYCLE COVERAGE & PERMISSIONS

- Android 13+ Consent: Requests POST_NOTIFICATIONS permission at runtime; handles denial with non-blocking toast.
- Foreground State: onMessage captures notifications, updates badge count, and appends to Alerts inbox.
- Background State: onBackgroundMessage runs isolated top-level handler saving record to Firestore inbox.
- Terminated State: getInitialMessage launches app from cold start; onMessageOpenedApp triggers deep link.

INTERACTIVE INBOX & DEEP LINKING

- In-App Notification Center: Alerts tab displays live history of received system updates and team invites.
- Interactive Actions: Decline invite includes non-blocking Undo snackbar that restores state immediately.
- Targeted Navigation: Tapping a notification with teamId payload navigates directly to Team Detail screen.
- Console Proof: Test messages sent from Firebase Console received across all 3 lifecycle states.

Google Analytics for Firebase & DebugView Verification

PRIVACY-BY-DESIGN EVENT PLAN (docs/analytics-plan.md)

- [x] Zero PII Policy: Absolutely no student email, full name, or raw student code passed as parameters.
- [x] User Identity: Authenticated User ID set once via setUserId(uid) following Google Sign-In.
- [x] Standard Recommended Names: login, logout, search_teams, view_team.
- [x] Custom Domain Events: join_team, cast_vote, invite_pool, lock_roster, export_report.
- [x] Typed Parameters: query_length (int), result_count (int), team_id (string), file_size_kb (int).

DEBUGVIEW REAL-TIME VERIFICATION

- [x] ADB Debug Flag: adb shell setprop debug.firebase.analytics.app com.devorganize222.exematching
- [x] Live Stream: All 9 events observed in real-time on Firebase Console -> Analytics -> DebugView.
- [x] Product Answers: Analytics answer key questions (e.g. Which topics are searched most? What % of teams lock?).
- [x] Evidence Saved: Detailed parameter inspector screenshots preserved in evidence/debugview/.

Crashlytics Telemetry & Real-Time Remote Config

FIREBASE CRASHLYTICS (firebase_crash_reporter.dart)

- [x] Fatal Crash Capture: FlutterError.onError and PlatformDispatcher record uncaught engine exceptions.
- [x] Handled Non-Fatal: Profile button triggers diagnostic recordError with custom keys (screen=profile, cohort=FA26).
- [x] Test Crash Action: Dedicated Profile button invokes .crash() -> verified on Console with User ID.
- [x] User Feedback: Non-fatal errors always display user-facing explanations, never crashing the UI.

FIREBASE REMOTE CONFIG (ZERO-REBUILD UPDATES)

- [x] In-App Fallback: ConfigKeys.defaults provides safe defaults when offline (enable_waiting_pool=true).
- [x] Instant Activation: onConfigUpdated automatically activates published parameters in real time.
- [x] Observable Behavior: Changing enable_waiting_pool on Console instantly hides Waiting Pool card on Home.
- [x] Zero Rebuild: Verified live without stopping the app, reinstalling the APK, or modifying code.

Automated E2E Testing with Google ARTEMIS

ARTEMIS AI AGENT HARNESS (e2e_test/)

- Multimodal Driving: ARTEMIS drives real Android 16 emulator using Gemini Flash profile (~3-5s/step).
- 13 Comprehensive Test Cases: Covers all 3 user flows, error paths, native share sheet, and storage upload.
- 3-Run Reliability Rule: Every case runs 3 times independently (39 runs total) to eliminate flaky results.
- Ground-Truth Assertions: Asserts on Semantics accessibility tree and Logcat (never status=="SUCCESS" alone).

39-RUN EXECUTION BENCHMARK RESULTS

- Overall Pass Rate: 39/39 (100% Pass) across all 13 test cases after harness stabilization.
- Average Duration: ~24.0s per test case execution.
- Harness Improvements (Commit 9652bb4): Token refresh on 401, E2E cooldown to respect 15 RPM quota.
- Trace Evidence: Complete run logs, accessibility dumps, and final.png images stored in e2e_test/evidence/.

ARTEMIS Exploratory Testing Findings

25-MINUTE UNSCRIPTED PRO SESSION

- Autonomous Agent: Driven by Gemini Pro Profile with zero pre-scripted steps.
- Goal: Identify potential layout overflows, state synchronization bugs, and network edge cases.
- Stress Scenarios Tested: Rapid filter chip toggling, long topic text truncation, network cut during voting.
- Zero Fatal Crashes: Application handled all chaotic inputs without unexpected crashes or freeze.

OBSERVATIONS & REFINEMENTS MADE

- Layout Resilience: Text wrapping and ellipsis prevented overflow errors at both 360dp and 412dp viewports.
- Firestore Resilience: Disconnecting network during vote triggered clean retry UI without data loss.
- Usability Improvement Identified: Added percentage progress label to PDF upload bar based on session feedback.
- Trust Assessment: Multi-run verification proved that AI agents are highly effective for mobile exploratory QA.

AI-Assisted Code Review on PR #8 (Google Gemini)

AUTOMATED REVIEW WORKFLOW (.github/workflows/)

- Single-Request Review: gemini_review.py sends source diff to gemini-3.5-flash-lite (completed in 14s).
- 5 Line-Tied Findings: Spanning security, concurrency, and reliability categories.
- Engineering Decisions: 3 accepted and fixed; 2 rejected with documented technical reasoning.
- Traceable Commits: Each accepted fix is anchored to its own Git commit naming the finding ID.

KEY FINDINGS & AUDITABLE DECISIONS

- GR-01 (Security · Accepted · Commit c9543a3): Join rule allowed modifying team metadata. Widened to whitelist 8 fields.
- GR-02 (Concurrency · Rejected): AI suggested querying inside transaction. Rejected: client SDK does not support it; in-transaction re-read already guarantees safety.
- GR-03 (Reliability · Accepted · Commit ee6ec78): KeyError on missing Firebase CLI token replaced with clear guidance.
- GR-04 (Reliability · Accepted · Commit d6926d1): Added logging for Remote Config start-up fetch failure.

Challenges Encountered, Lessons Learned & Conclusion

ENGINEERING CHALLENGES & SOLUTIONS

- Challenge 1: Firebase Storage rules cannot perform cross-service firestore.get() lookups -> Solved via UID folder isolation.
- Challenge 2: Multi-turn AI review exhausted free-tier API quotas -> Solved via single-request structured review script.
- Challenge 3: Background isolate handling of FCM messages -> Solved with dedicated top-level background handler.
- Challenge 4: K19+ non-edu email accounts -> Solved with Centralized Roster Allowlist on Firestore.

FINAL VERIFICATION & CONCLUSION

- Architecture: 100% MVVM compliance with Provider; zero Firebase SDK calls in presentation widgets.
- Firebase: All 6 services verified with live device evidence, security rules, and real-time synchronization.
- Testing: 13 E2E test cases verified across 39 runs with Google ARTEMIS + 25-minute exploratory testing.
- Conclusion: EXEMatching is a robust, cloud-enabled production-ready Flutter app meeting all Lab 3 criteria.